



codemanship

Refactoring

Workbook

Jason Gorman

Essential Code Craft

Contents

Get The Code	2
Getting Started	3
Test && Commit Revert (TCR)	5
Part I – Code Smells	6
Code Smell: Comments	6
Code Smell: Long Method	7
Code Smell: Divergent Change	7
Code Smell: Feature Envy & Data Class	8
Code Smell: Duplication	9
Code Smell: Long Parameter Lists & Data Clumps	11
Part II – Legacy Code	13
The Mikado Method	14
APPENDIX - Refactoring Guides	17
1. Rename (Find & Replace)	17
2. Extract Method	20
3. Inline Method	23
4. Move Method	26
5. Extract Class	31
6. Introduce Parameter Object	37
7. Extract Interface	42
There Are Many More Refactorings Than These	47

Get The Code

Python

https://github.com/jasongorman/python_refactoring_essential

C#

https://github.com/jasongorman/csharp_refactoring_essential

Java

https://github.com/jasongorman/java_refactoring_essential

Kotlin

https://github.com/jasongorman/kotlin_refactoring_essential

JavaScript

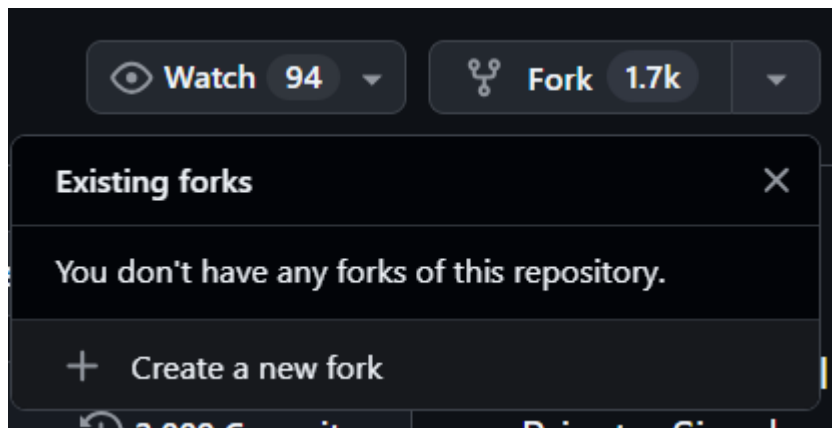
https://github.com/jasongorman/js_refactoring_essential

TypeScript

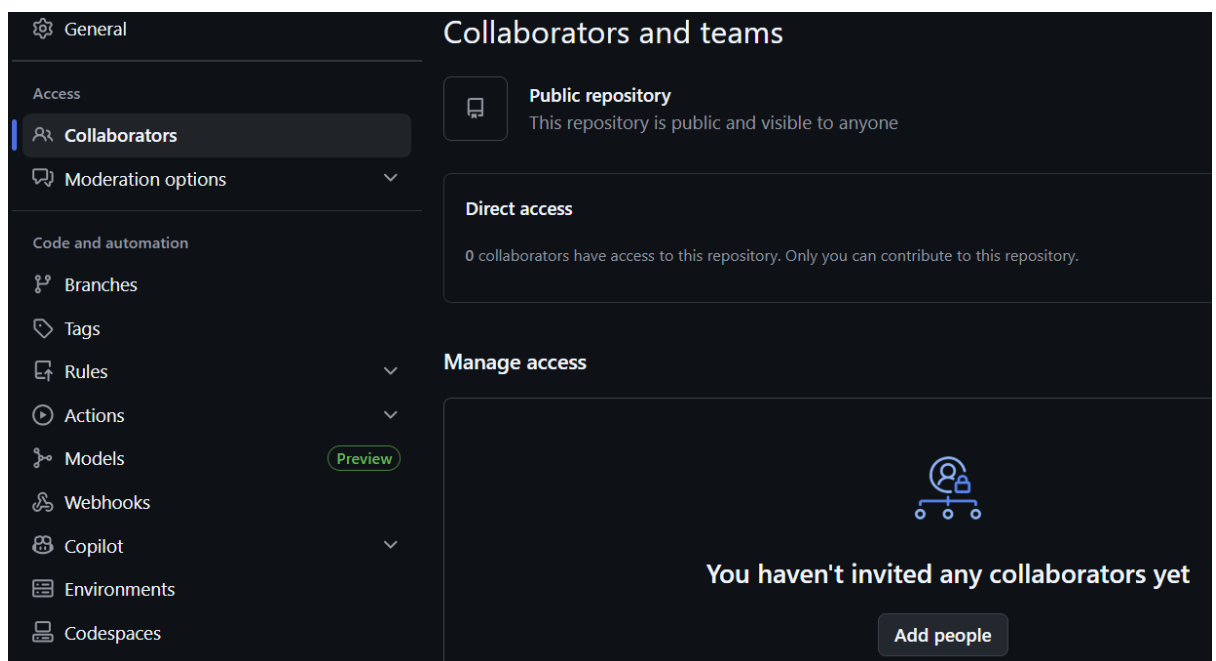
https://github.com/jasongorman/ts_refactoring_essential

Getting Started

1. Select the project you want to use
2. Fork the GitHub repository



3. Invite your pairing partner to collaborate (partner – check your emails)



4. Enable the included GitHub CI workflow to build and test the project whenever one of you pushes to the main branch.



Workflows aren't being run on this forked repository

Because this repository contained workflow files when it was forked, we have disabled them from running on this fork. Make sure you understand the configured workflows and their expected usage before enabling Actions on this repository.

[I understand my workflows, go ahead and enable them](#)

[View the workflows directory](#)

5. Clone the repo to your local hard drive.
6. Open the project and RUN THE TESTS

Test & Commit || Revert (TCR)

TCR is a useful technique to help us calibrate how many changes to the code in each refactoring step are too many – too likely to break something.

In the root of your project folder, you will find a file called `tcr.bat` for Windows. You can adapt it for other platforms, such as Bash. Just as your favourite LLM – e.g., ChatGPT – to translate it for you.

If you want to have a go at TCR, run your tests using the batch script instead of your usual way. If all the tests pass, your changes will be automatically committed. If any tests fail (or some error occurs), your changes will automatically be reverted to the previous successful commit.

Part I – Code Smells

A code smell is the name given to signs of increasing disorder in our code – a sign that the internal design is becoming rigid (hard to change) and brittle (easy to break). Without continuous refactoring, code smells build up and the cost of changing the code increases until change becomes too risky and expensive. Refactoring is an essential ongoing part of any development process.

Code Smell: Comments

Comments become a code smell when they are used to explain *what the code already clearly expresses*. In the example, most comments simply restate the obvious: looping over a range, squaring a number, or returning a value. This creates noise rather than clarity.

The core issue isn't that comments are bad in general, but that **they are compensating for code that could be self-explanatory**. Over time, these comments can become misleading if the code changes but the comments don't, and they add cognitive overhead for readers who must read both code and commentary to understand the logic.

In this example, the method names `m` and `n`, along with single-letter variables like `p`, `q`, `z`, and `k`, force the reader to rely on comments to understand intent. That's usually a sign the code itself isn't communicating well enough.

Refactoring Approaches

The goal here is to remove the need for comments by making the code clearer and more expressive.

- **Rename Method**
Replace unclear method names like `m` and `n` with intention-revealing names (e.g., `sumOfSquares`, `square`). This often eliminates the need for explanatory comments entirely.
- **Rename Variable**
Improve readability by replacing single-letter variables (`p`, `q`, `z`, `k`) with meaningful names like `sum`, `start`, `end`, and `value`.
- **Extract Method**
If logic becomes more complex during refactoring, extract meaningful steps into well-named methods (e.g., `square(int value)` or `sumRangeSquares(int start, int end)`), making the flow readable without comments.
- **Inline Method** (where appropriate)
If a method like `square` is trivial, consider inlining it to reduce unnecessary abstraction - especially if it improves readability in context.

Overall, the aim is to shift understanding from *"I need to read the comments to understand this code"* to *"the code explains itself."*

Code Smell: Long Method

A long method becomes a code smell when it tries to do too many things in one place. In this example, `summarise()` handles validation, pricing, discount rules, tax calculation, and final aggregation all in a single block of code.

The problem isn't just length - it's **lack of separation of concerns**. Each section represents a distinct responsibility, but because they're all bundled together, the method becomes harder to read, test, and maintain. It also increases the cognitive load: you have to mentally track multiple steps and intermediate values (subtotal, discount, tax, total) at once.

Long methods also tend to grow over time. New rules get appended rather than structured, making the method increasingly fragile and difficult to modify safely.

Refactoring Approaches

The main goal is to break the method into smaller, well-named pieces that each do one thing clearly.

- **Extract Method**

Break each logical section into its own method:

- `validateItems()`
- `calculateSubtotal()`
- `calculateDiscount(double subtotal)`
- `calculateTax(double taxableAmount)`
- `calculateTotal(double taxableAmount, double tax)`

This is the primary refactoring for long methods and immediately improves readability and testability.

- **Move Method (if extraction reveals better ownership)**

If calculations like discount rules or tax logic grow more complex, they may belong on `Customer`, `Order`, or a `pricing/service` class rather than staying in `Order`.

Overall, the refactoring goal is to turn `summarise()` from a *process dump* into a *composed method* that delegates clear, focused responsibilities to smaller methods. A general principle of good modular software design is *complexity by composition*.

Code Smell: Divergent Change

Divergent Change occurs when a single class has to change for many different reasons. In this example, `CustomerService` contains multiple unrelated responsibilities: email validation, name formatting, loyalty points calculation, and account status rules.

The key issue is that these behaviours **do not vary together**. If email validation rules change, you modify this class. If loyalty logic changes, you modify the same class. If display formatting rules change, again the same class is affected. Over time, this leads to a class that is frequently edited for unrelated reasons, increasing the risk of unintended side effects.

This violates the **Single Responsibility Principle**, because the class has multiple axes of change rather than a single coherent purpose. It becomes harder to maintain, harder to test in isolation, and more likely to accumulate accidental complexity.

Refactoring Approaches

The main goal is to separate the class into focused components, each with a single reason to change.

- **Extract Class**

Split responsibilities into dedicated classes based on domain concern:

- EmailValidator → email validation logic
- CustomerDisplayFormatter → name formatting
- LoyaltyService → loyalty points calculation
- AccountStatusService → account status rules

This is the primary refactoring for Divergent Change.

- **Move Method**

Move each method into the most appropriate new class so behaviour lives with its relevant responsibility.

Overall, the refactoring goal is to transform CustomerService from a *miscellaneous utility bucket* into a set of focused, single-purpose classes where each area of behaviour changes independently.

Code Smell: Feature Envy & Data Class

This example actually shows two related code smells working together: **Feature Envy** and **Data Class**. They often appear in the same design and reinforce each other.

Feature Envy

Feature Envy occurs when a method is more focused on another object's data than its own responsibilities. Here, PriceCalculator.calculateFinalPrice() is essentially driven entirely by the state of Product.

It repeatedly queries Product (getPrice(), isOnSale()) and contains all the logic for how a Product should behave when pricing is applied. This suggests that the behaviour is sitting in the wrong place.

The problem is not just stylistic - it increases coupling. PriceCalculator becomes tightly dependent on the internal structure of Product, and any change to pricing-related data in Product ripples outward into external logic.

Data Class (Product)

Product is also a **Data Class** because it appears to primarily expose state (via getters) without meaningful behaviour. It holds data, but does not “own” the logic that naturally belongs with it.

Data Classes are problematic because they:

- Encourage procedural code elsewhere (like PriceCalculator)
- Break encapsulation by exposing internal state
- Lead to scattered business logic across multiple service classes

In a healthy object-oriented design, objects should combine **data + behaviour**, not just act as passive structures.

Refactoring Approaches

The key goal is to reunite data and behaviour while improving cohesion.

- **Move Method** (primary refactoring)
Move calculateFinalPrice into Product, so pricing becomes behaviour of the product itself:
 - product.calculateFinalPrice()
 - **Encapsulate Data**
Gradually reduce direct exposure of internal state. Instead of exposing getters like getPrice() and isOnSale(), prefer meaningful behaviour (e.g. isDiscounted(), getFinalPrice()). This style of design is called “Tell, Don’t Ask”.
-

Summary

Feature Envy here is a symptom of the deeper issue: **Product is under-behaved (Data Class), so logic leaks into external classes**. Fixing one naturally helps resolve the other. The ideal end state is a Product that both *holds data and knows how to act on it*.

Code Smell: Duplication

Duplication becomes a code smell when the same or very similar logic appears in multiple places without a clear structural reason. In this example, the formatting logic for each time-of-day (Morning, Afternoon, Evening, Night) is almost identical, differing only in a label.

At first glance, this might seem harmless, but duplication creates **hidden maintenance risk**: if the format changes (e.g. adding humidity or changing the temperature format), the change must be repeated in multiple places. This increases the chance of inconsistency and bugs.

That said, duplication is not always a design failure - it can be a *signal*. Sometimes it indicates that the code is missing an abstraction or that responsibilities have been split in a suboptimal way.

Rule of Three

A useful guideline here is the **Rule of Three**:

The first time you write something, you just write it.

The second time, you tolerate duplication.

The third time, you refactor.

This rule avoids premature abstraction. Not all duplication should be eliminated immediately - some duplication is temporary and helps you understand the real domain shape before introducing abstractions.

DRY vs Readability

While **DRY (Don't Repeat Yourself)** is a valuable principle, it should not override clarity. In many cases, *readability wins over DRY*. Over-aggressive deduplication can lead to overly abstract code that is harder to understand than the original repetition.

In this example, the repeated string construction is simple and explicit. A poorly chosen abstraction (e.g. a complex formatter hierarchy) could actually make the code harder to follow.

The best design is the one that makes intent clearest to the reader, even if that means accepting some duplication.

Duplication as a Design Signal

From a more evidence-based architecture perspective, duplication can be a **diagnostic tool rather than a problem in itself**. Reuse is effectively the opposite of duplication, but forced reuse can sometimes create the wrong abstractions too early.

Here, the repetition of nearly identical formatting logic across time-of-day branches suggests one of two possibilities:

- There is a missing concept like a `TimeOfDayFormatter`
- Or the variation is not significant enough to justify abstraction yet

Duplication therefore helps reveal where modular boundaries *might* exist, rather than dictating them immediately.

Refactoring Approaches

The goal is to remove harmful repetition while avoiding unnecessary abstraction.

- **Extract Method**
Pull out the repeated formatting logic into a single method, e.g. `formatLine(label, forecast)`, which captures the shared structure.
- **Introduce Parameter Object** (light use here)
Represent time-of-day as a value object or enum that carries its label and behaviour, rather than four separate if blocks.
- **Move Method**
If formatting logic belongs more naturally with `Forecast` or a dedicated formatter class, relocate it to improve cohesion.

Summary

Duplication is not simply “bad code” - it is often a **clue**. The key is to apply the **Rule of Three**, prioritise **readability over premature DRY**, and treat repetition as evidence that can guide better modular design decisions rather than a problem to eliminate immediately.

Code Smell: Long Parameter Lists & Data Clumps

A Long Parameter List becomes a code smell when a method requires too many inputs to do its job. In this example, `generateShippingNote` takes ten separate parameters spanning customer details, address details, and order details.

The core issue is not just verbosity - it's that the method signature becomes **hard to read, hard to use, and easy to misuse**. Callers must remember parameter order, and the method becomes brittle: any change to input structure forces changes across all call sites.

They're often a sign that the method or function has too many responsibilities, and Long Parameter Lists have a tendency to get longer, breaking compatibility with client code each time.

Even more importantly, this example contains **Data Clumps**: groups of parameters that naturally belong together but are passed around separately. Here, we can clearly see:

- Customer data: `customerFirstName`, `customerLastName`
- Address data: `addressLine1`, `addressLine2`, `city`, `postcode`, `country`
- Order data: `orderId`, `itemDescription`, `quantity`

When the same groups of data repeatedly appear together, it usually signals that a missing abstraction exists.

Why This Matters

Long parameter lists often indicate that responsibilities are not well-encapsulated. Instead of passing structured objects that represent real concepts, the code passes raw primitives, losing meaning along the way.

This leads to:

- Reduced readability (what does each String represent?)
- Higher risk of parameter mis-ordering
- Poor cohesion (logic operates on scattered data rather than meaningful objects)
- Difficulty extending behaviour (new fields require signature changes everywhere)

Refactoring Approaches

The main goal is to replace scattered primitives with meaningful domain structures and reduce the method's surface area.

- **Introduce Parameter Object** (primary refactoring)
Replace data clumps with cohesive objects:
 - Customer (first name, last name)
 - Address (address lines, city, postcode, country)
 - Order or OrderItem (orderId, itemDescription, quantity)

The method signature becomes far simpler and more intention-revealing.

- **Move Method** (if logic applied to a parameter object really belongs with that object)

Summary

Long Parameter Lists are often a surface-level symptom of a deeper issue: missing domain structure. The presence of **Data Clumps** is a strong signal that related data is being modelled incorrectly as separate primitives.

Refactoring here is about restoring meaning - replacing raw parameters with cohesive objects that represent real concepts in the domain, and in which related logic can be encapsulated.

Part II – Legacy Code

In your project, you will find a package or folder with “legacy code” in it.

What makes that code legacy code? When code is risky and/or expensive to change, we say it’s legacy code. And what makes this particular code risky to change is that there are **no fast-running automated tests** for it.

In this second part of the workshop, you are charged with making a change to the logic of calculating shipping.

Right now, there are three types of shipping – STANDARD, EXPRESS and OVERNIGHT.

We have a new requirement for a fourth type of shipping – **INTERNATIONAL**.

In the project, you will find a *test.bat* file that I’ve been using to manually inspect calculations in the terminal for three orders that cover the different types of shipping – 1001, 1002 and 1003.

There’s a fourth order – 1004 – that has the new INTERNATIONAL shipping type.

Shipping for INTERNATIONAL orders is calculated as **order.weightKg * 1.5**

Being responsible developers, we wish to add this new behaviour by writing a failing test for it first, and we would wish to ensure that any changes we make don’t break existing behaviour.

This is how you should approach this change in the exercise:

1. Identify where is the code that needs to change
2. Identify how you would test the existing code to ensure it still works
3. *Carefully* refactor the ShippingCalculator class specifically to make it easy write to unit tests for. While you’re doing this – as with all refactoring – you should retest the shipping calculation for the existing cases. You can use the test.bat file – or your .sh or other version of it – to manually inspect the results in your terminal. DO NOT SKIP TESTING, even if it takes a lot of time. (The alternative can cost even more time, believe me!)
4. *Write those unit tests to reproduce your manual tests!*
5. In a test-driven way, make the change you’ve been asked to make.
6. Now you have fast automated tests for that code, take advantage and apply “The Boy Scout Rule” (always leave the campsite tidier than you found it). Focus on that switch statement. We had to add another case to it for a new type of shipping. Odds are, there’ll be more in the future – so we may have to keep changing that class for each new one. That breaks the Open-Closed Principle (OCP) – classes should be open to extension and closed to modification.

This final refactoring is what we call *long-form* – it’s not just a simple Rename or Extract Method. The name of this long-form refactoring is Replace Conditional With Polymorphism, or more specifically Replace Type Code With Strategy.

You can execute this refactoring as a sequence of smaller, safer refactorings like Extract Method, Extract Class and Extract Interface.

You may find it helpful to roughly map out the sequence of refactorings you’re planning to do to give you some high-level guidance.

The Mikado Method

Here's a simple technique for breaking complex refactorings down and building a high-level roadmap to guide you.

Let's say we want to replace a multi-case conditional with a polymorphic design that will enable us to add new cases without touching the original code.

We want to turn this design:

```
def calculate(price, customer_type):  
    if customer_type == "regular":  
        return price * 1.0  
    elif customer_type == "premium":  
        return price * 0.9  
    elif customer_type == "vip":  
        return price * 0.8  
    else:  
        raise ValueError("Unknown type")
```

Into this:

```
class PricingStrategy(ABC):  
    @abstractmethod  
    def calculate(self, price: float) -> float:  
        pass  
  
class RegularPricing(PricingStrategy):  
    def calculate(self, price: float) -> float:  
        return price * 1.0  
  
class PremiumPricing(PricingStrategy):  
    def calculate(self, price: float) -> float:  
        return price * 0.9
```

```

class VIPPricing(PricingStrategy):
    def calculate(self, price: float) -> float:
        return price * 0.8

class PricingStrategyFactory:
    _strategies = {
        "regular": RegularPricing,
        "premium": PremiumPricing,
        "vip": VIPPricing,
    }

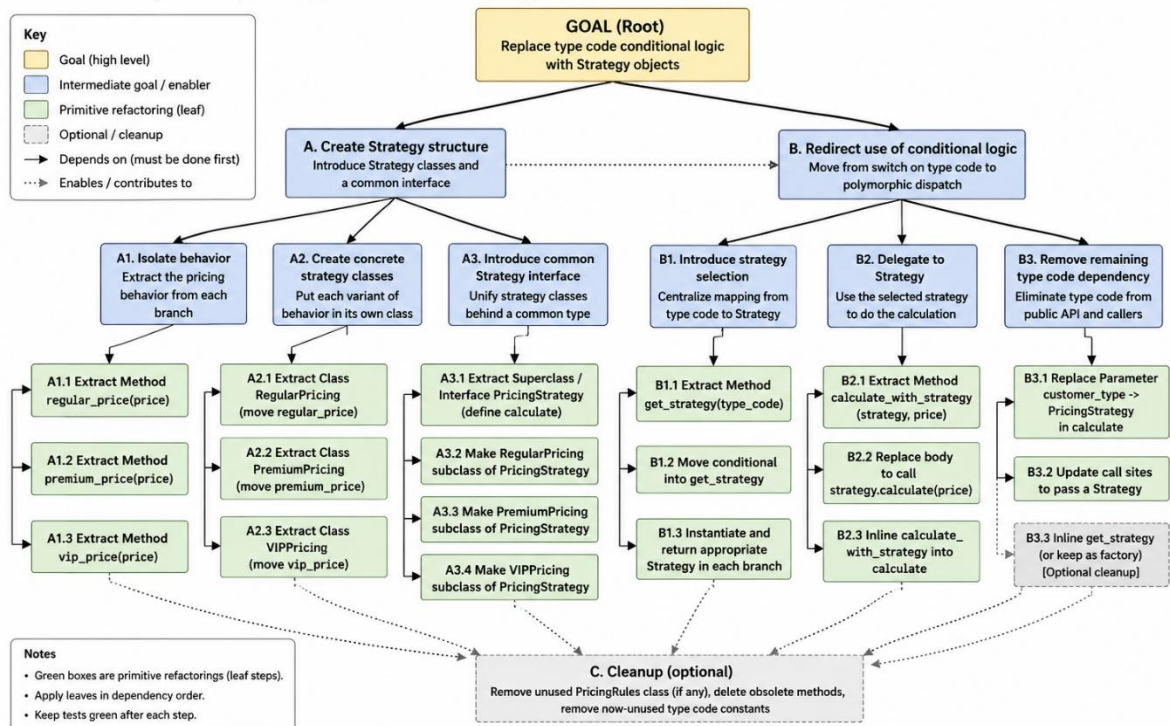
    @staticmethod
    def create(customer_type: str) -> PricingStrategy:
        try:
            strategy_cls = PricingStrategyFactory._strategies[customer_type]
            return strategy_cls()
        except KeyError:
            raise ValueError(f"Unknown customer type: {customer_type}")

    def calculate(price: float, customer_type: str) -> float:
        strategy = PricingStrategyFactory.create(customer_type)
        return strategy.calculate(price)

```

Grab a (virtual) sheet of paper and write your goal on it, leaving plenty of space for sub-goals. Then break those sub-goals down further, until you have a map of primitive refactorings you can execute as a sequence safely.

Mikado Graph – Replace Type Code with Strategy



APPENDIX - Refactoring Guides

1. Rename (Find & Replace)

Purpose

Rename improves clarity by replacing an old name with a better one. In this workbook, we demonstrate the mechanical approach using **Find & Replace**:

- Replace method name everywhere it appears
- Update both definition and call sites
- Avoid replacing unintended text (comments/strings)

Example

Rename: calc → calculate

Java

Before

```
class Calculator {  
    int calc(int a, int b) {  
        return a + b;  
    }  
  
    void run() {  
        int result = calc(2, 3);  
        System.out.println(result);  
    }  
}
```

After

```
class Calculator {  
    int calculate(int a, int b) {  
        return a + b;  
    }  
  
    void run() {  
        int result = calculate(2, 3);  
        System.out.println(result);  
    }  
}
```

JavaScript

Before

```
function calc(a, b) {  
    return a + b;  
}
```

```
console.log(calc(2, 3));
```

After

```
function calculate(a, b) {  
    return a + b;  
}
```

```
console.log(calculate(2, 3));
```

C#

Before

```
class Calculator  
{  
    int calc(int a, int b)  
    {  
        return a + b;  
    }  
  
    void Run()  
    {  
        int result = calc(2, 3);  
        Console.WriteLine(result);  
    }  
}
```

After

```
class Calculator  
{  
    int calculate(int a, int b)  
    {  
        return a + b;  
    }  
  
    void Run()  
    {  
        int result = calculate(2, 3);  
        Console.WriteLine(result);  
    }  
}
```

Python

Before

```
def calc(a, b):  
    return a + b  
  
print(calc(2, 3))
```

After

```
def calculate(a, b):  
    return a + b  
  
print(calculate(2, 3))
```

Checklist

- All occurrences updated (definition + call sites)
- No accidental replacements in comments/strings
- Tests still pass after rename
- Name is clearer and more intention-revealing

2. Extract Method

Purpose

Extract Method improves readability by taking a **fragment of code inside a method** and moving it into a new method with a meaningful name.

Use when:

- A method is doing more than one thing
- You want to name a logical step in a process
- You want to reduce duplication or complexity

Transformation

- Identify a code block
- Move it into a new method
- Replace the block with a method call

Example

We extract the logic that calculates a total from a larger method.

Java

Before

```
class Order {  
    void printReceipt() {  
        int price1 = 10;  
        int price2 = 20;  
  
        int total = price1 + price2;  
  
        System.out.println("Total: " + total);  
    }  
}
```

After

```
class Order {  
    void printReceipt() {  
        int total = calculateTotal(10, 20);  
        System.out.println("Total: " + total);  
    }  
  
    int calculateTotal(int price1, int price2) {  
        return price1 + price2;  
    }  
}
```

```
}  
}
```

JavaScript

Before

```
function printReceipt() {  
  const price1 = 10;  
  const price2 = 20;  
  
  const total = price1 + price2;  
  
  console.log("Total: " + total);  
}
```

After

```
function printReceipt() {  
  const total = calculateTotal(10, 20);  
  console.log("Total: " + total);  
}  
  
function calculateTotal(price1, price2) {  
  return price1 + price2;  
}
```

C#

Before

```
class Order  
{  
  void PrintReceipt()  
  {  
    int price1 = 10;  
    int price2 = 20;  
  
    int total = price1 + price2;  
  
    Console.WriteLine("Total: " + total);  
  }  
}
```

After

```
class Order  
{  
  void PrintReceipt()
```

```
{
    int total = CalculateTotal(10, 20);
    Console.WriteLine("Total: " + total);
}

int CalculateTotal(int price1, int price2)
{
    return price1 + price2;
}
}
```

Python

Before

```
def print_receipt():
    price1 = 10
    price2 = 20

    total = price1 + price2

    print("Total:", total)
```

After

```
def print_receipt():
    total = calculate_total(10, 20)
    print("Total:", total)

def calculate_total(price1, price2):
    return price1 + price2
```

Checklist

- Extracted code represents a single logical task
- New method has a clear, intention-revealing name
- Original method is simpler and easier to read
- Behaviour remains unchanged after refactoring
- Tests still pass

3. Inline Method

Purpose

Inline Method is the inverse of Extract Method. It removes a method whose body is as clear as its name, and replaces calls to it with the method's contents.

Use when:

- The method body is trivial
- The method name adds no extra clarity
- The method is only a simple pass-through

Transformation

- Replace method call with method body
- Delete the method definition
- Ensure behaviour remains unchanged

Example

We inline a simple method `calculateTotal` that adds two numbers.

Java

Before

```
class Order {  
    void printReceipt() {  
        int total = calculateTotal(10, 20);  
        System.out.println("Total: " + total);  
    }  
  
    int calculateTotal(int a, int b) {  
        return a + b;  
    }  
}
```

After

```
class Order {  
    void printReceipt() {  
        int total = 10 + 20;  
        System.out.println("Total: " + total);  
    }  
}
```

JavaScript

Before

```
function printReceipt() {  
  const total = calculateTotal(10, 20);  
  console.log("Total: " + total);  
}  
  
function calculateTotal(a, b) {  
  return a + b;  
}
```

After

```
function printReceipt() {  
  const total = 10 + 20;  
  console.log("Total: " + total);  
}
```

C#

Before

```
class Order  
{  
  void PrintReceipt()  
  {  
    int total = CalculateTotal(10, 20);  
    Console.WriteLine("Total: " + total);  
  }  
  
  int CalculateTotal(int a, int b)  
  {  
    return a + b;  
  }  
}
```

After

```
class Order  
{  
  void PrintReceipt()  
  {  
    int total = 10 + 20;  
    Console.WriteLine("Total: " + total);  
  }  
}
```

Python

Before

```
def print_receipt():  
    total = calculate_total(10, 20)  
    print("Total:", total)
```

```
def calculate_total(a, b):  
    return a + b
```

After

```
def print_receipt():  
    total = 10 + 20  
    print("Total:", total)
```

Checklist

- Method body is simple enough to inline safely
- All call sites replaced with method body
- Method definition removed
- No loss of clarity in the resulting code
- Tests still pass

4. Move Method

Purpose

Move Method improves design by relocating behaviour to the class that owns the data it uses.

Typical smell: *Feature Envy*

A method in one class spends more time using another class's data than its own.

Use when:

- A method repeatedly accesses another object's fields
- Behaviour clearly belongs with the data it operates on
- Moving the method increases cohesion

Safe manual mechanics (no automation)

When moving a method manually:

1. Identify the method's real "home" (the data it uses most)
2. Copy the method into the target class
3. Replace references to the old object with parameters or this
4. Adjust visibility (public/private as needed)
5. Update the original method to delegate (temporary step)
6. Update all call sites
7. Delete the original once safe
8. Run tests after each step

Key idea: behaviour should "follow the data"

Example

Scenario

OrderService calculates a discount using data from Order.

This is **Feature Envy**: the logic belongs on Order.

We move: `calculateDiscount(Order)` → `Order.calculateDiscount()`

Java

Before

```

class Order {
    double total;
    int loyaltyYears;

    Order(double total, int loyaltyYears) {
        this.total = total;
        this.loyaltyYears = loyaltyYears;
    }
}

class OrderService {
    double calculateDiscount(Order order) {
        if (order.loyaltyYears > 5) {
            return order.total * 0.2;
        }
        return order.total * 0.1;
    }
}

```

After

```

class Order {
    double total;
    int loyaltyYears;

    Order(double total, int loyaltyYears) {
        this.total = total;
        this.loyaltyYears = loyaltyYears;
    }

    double calculateDiscount() {
        if (loyaltyYears > 5) {
            return total * 0.2;
        }
        return total * 0.1;
    }
}

class OrderService {
    double calculateDiscount(Order order) {
        return order.calculateDiscount();
    }
}

```

JavaScript

Before

```
class Order {
    constructor(total, loyaltyYears) {
        this.total = total;
        this.loyaltyYears = loyaltyYears;
    }
}
```

```
class OrderService {
    calculateDiscount(order) {
        if (order.loyaltyYears > 5) {
            return order.total * 0.2;
        }
        return order.total * 0.1;
    }
}
```

After

```
class Order {
    constructor(total, loyaltyYears) {
        this.total = total;
        this.loyaltyYears = loyaltyYears;
    }

    calculateDiscount() {
        if (this.loyaltyYears > 5) {
            return this.total * 0.2;
        }
        return this.total * 0.1;
    }
}

class OrderService {
    calculateDiscount(order) {
        return order.calculateDiscount();
    }
}
```

C#

Before

```
class Order
{
    public double Total;
    public int LoyaltyYears;

    public Order(double total, int loyaltyYears)
```

```

    {
        Total = total;
        LoyaltyYears = loyaltyYears;
    }
}

class OrderService
{
    public double CalculateDiscount(Order order)
    {
        if (order.LoyaltyYears > 5)
        {
            return order.Total * 0.2;
        }
        return order.Total * 0.1;
    }
}

```

After

```

class Order
{
    public double Total;
    public int LoyaltyYears;

    public Order(double total, int loyaltyYears)
    {
        Total = total;
        LoyaltyYears = loyaltyYears;
    }

    public double CalculateDiscount()
    {
        if (LoyaltyYears > 5)
        {
            return Total * 0.2;
        }
        return Total * 0.1;
    }
}

class OrderService
{
    public double CalculateDiscount(Order order)
    {
        return order.CalculateDiscount();
    }
}

```

Python

Before

```
class Order:
    def __init__(self, total, loyalty_years):
        self.total = total
        self.loyalty_years = loyalty_years
```

```
class OrderService:
    def calculate_discount(self, order):
        if order.loyalty_years > 5:
            return order.total * 0.2
        return order.total * 0.1
```

After

```
class Order:
    def __init__(self, total, loyalty_years):
        self.total = total
        self.loyalty_years = loyalty_years
```

```
    def calculate_discount(self):
        if self.loyalty_years > 5:
            return self.total * 0.2
        return self.total * 0.1
```

```
class OrderService:
    def calculate_discount(self, order):
        return order.calculate_discount()
```

Checklist

- Method moved to the class that owns the most-used data
- No direct field access from the old location remains
- Call sites updated to use new location
- Temporary delegation removed when stable
- Behaviour unchanged
- Tests still pass

5. Extract Class

Purpose

Extract Class improves design by splitting a class that does too much into two (or more) focused classes.

Use when:

- A class has multiple responsibilities
- A subset of fields and methods form a coherent group
- The class is becoming difficult to understand or maintain

Safe manual mechanics (no automation)

Extract Class should be done in **small, testable steps** to avoid breaking behaviour.

Step-by-step process

1. **Identify the responsibility to extract**
 - Group related fields and methods
2. **Declare the new class**
 - Start empty
3. **Move fields first (optional but common)**
 - Copy fields into the new class
 - Provide constructor parameters if needed
4. **Move methods gradually**
 - Copy one method at a time
 - Update field access to refer to the new class
5. **Create an instance in the original class**
 - As a field or local variable
6. **Retarget method calls**
 - Replace old logic with calls to the new class instance
7. **Run tests after each small move**
8. **Remove duplicated code from original class**
 - Once fully migrated

Key principle: keep the system working after each micro-step

Example

Scenario

Order contains both order data and discount logic. We extract discount logic into DiscountCalculator.

Java

Before

```
class Order {
    double total;
    int loyaltyYears;

    Order(double total, int loyaltyYears) {
        this.total = total;
        this.loyaltyYears = loyaltyYears;
    }

    double calculateDiscount() {
        if (loyaltyYears > 5) {
            return total * 0.2;
        }
        return total * 0.1;
    }

    double finalPrice() {
        return total - calculateDiscount();
    }
}
```

After

```
class DiscountCalculator {
    double calculateDiscount(double total, int loyaltyYears) {
        if (loyaltyYears > 5) {
            return total * 0.2;
        }
        return total * 0.1;
    }
}

class Order {
    double total;
    int loyaltyYears;
    DiscountCalculator discountCalculator = new DiscountCalculator();

    Order(double total, int loyaltyYears) {
```

```

        this.total = total;
        this.loyaltyYears = loyaltyYears;
    }

    double calculateDiscount() {
        return discountCalculator.calculateDiscount(total, loyaltyYears);
    }

    double finalPrice() {
        return total - calculateDiscount();
    }
}

```

JavaScript

Before

```

class Order {
    constructor(total, loyaltyYears) {
        this.total = total;
        this.loyaltyYears = loyaltyYears;
    }

    calculateDiscount() {
        if (this.loyaltyYears > 5) {
            return this.total * 0.2;
        }
        return this.total * 0.1;
    }

    finalPrice() {
        return this.total - this.calculateDiscount();
    }
}

```

After

```

class DiscountCalculator {
    calculateDiscount(total, loyaltyYears) {
        if (loyaltyYears > 5) {
            return total * 0.2;
        }
        return total * 0.1;
    }
}

class Order {
    constructor(total, loyaltyYears) {

```

```

    this.total = total;
    this.loyaltyYears = loyaltyYears;
    this.discountCalculator = new DiscountCalculator();
}

calculateDiscount() {
    return this.discountCalculator.calculateDiscount(this.total, this.loyaltyYears);
}

finalPrice() {
    return this.total - this.calculateDiscount();
}
}

```

C#

Before

```

class Order
{
    double Total;
    int LoyaltyYears;

    public Order(double total, int loyaltyYears)
    {
        Total = total;
        LoyaltyYears = loyaltyYears;
    }

    public double CalculateDiscount()
    {
        if (LoyaltyYears > 5)
        {
            return Total * 0.2;
        }
        return Total * 0.1;
    }

    public double FinalPrice()
    {
        return Total - CalculateDiscount();
    }
}

```

After

```

class DiscountCalculator
{

```

```

public double CalculateDiscount(double total, int loyaltyYears)
{
    if (loyaltyYears > 5)
    {
        return total * 0.2;
    }
    return total * 0.1;
}
}

class Order
{
    double Total;
    int LoyaltyYears;
    DiscountCalculator discountCalculator = new DiscountCalculator();

    public Order(double total, int loyaltyYears)
    {
        Total = total;
        LoyaltyYears = loyaltyYears;
    }

    public double CalculateDiscount()
    {
        return discountCalculator.CalculateDiscount(Total, LoyaltyYears);
    }

    public double FinalPrice()
    {
        return Total - CalculateDiscount();
    }
}

```

Python

Before

```

class Order:
    def __init__(self, total, loyalty_years):
        self.total = total
        self.loyalty_years = loyalty_years

    def calculate_discount(self):
        if self.loyalty_years > 5:
            return self.total * 0.2
        return self.total * 0.1

```

```
def final_price(self):
    return self.total - self.calculate_discount()
```

After

```
class DiscountCalculator:
    def calculate_discount(self, total, loyalty_years):
        if loyalty_years > 5:
            return total * 0.2
        return total * 0.1

class Order:
    def __init__(self, total, loyalty_years):
        self.total = total
        self.loyalty_years = loyalty_years
        self.discount_calculator = DiscountCalculator()

    def calculate_discount(self):
        return self.discount_calculator.calculate_discount(
            self.total,
            self.loyalty_years
        )

    def final_price(self):
        return self.total - self.calculate_discount()
```

Checklist

- Responsibility clearly identified and isolated
- New class introduced before moving logic
- Fields moved gradually and safely
- Methods migrated one at a time
- Original class delegates to new class
- No duplicated logic remains
- Tests still pass after each step

6. Introduce Parameter Object

Purpose

Introduce Parameter Object replaces a group of related parameters with a single object. This improves readability, reduces parameter repetition, and makes methods easier to extend.

Use when:

- A method has several related parameters
- The same parameter groups appear across multiple methods
- You want to model a concept explicitly (e.g. range, address, date window)

Safe manual mechanics (no automation)

This refactoring must be done **incrementally** to avoid breaking call sites.

Step-by-step process

1. **Declare the new class**
 - Create a new class to represent the parameter group
2. **Add properties via constructor**
 - Each original parameter becomes a property
 - Initialise all properties in the constructor
3. **Add the new parameter object alongside existing parameters**
 - Do NOT remove old parameters yet
 - Construct the new object at call sites using existing values
4. **Update method implementation**
 - Replace direct parameter usage with object properties
5. **Safe delete parameters one at a time**
 - Remove one old parameter per step
 - Update all call sites each time
 - Re-run tests after each removal

Safe Delete (important)

“Safe Delete” means removing a parameter only when you are sure nothing depends on it.

How to do it safely:

- Remove the parameter from the method signature

- Update all call sites immediately
- Ensure the parameter object fully replaces its usage
- Run tests before removing the next parameter

Rule: never delete multiple parameters at once in a manual refactor

Example

Scenario

A method takes a group of values representing a “date range”.

We introduce `DateRange` to encapsulate them.

Java

Before

```
class ReportService {
    void generateReport(int startDay, int startMonth, int endDay, int endMonth) {
        System.out.println("From " + startDay + "/" + startMonth +
            " to " + endDay + "/" + endMonth);
    }
}
```

After (introducing parameter object)

```
class DateRange {
    int startDay;
    int startMonth;
    int endDay;
    int endMonth;

    DateRange(int startDay, int startMonth, int endDay, int endMonth) {
        this.startDay = startDay;
        this.startMonth = startMonth;
        this.endDay = endDay;
        this.endMonth = endMonth;
    }
}

class ReportService {
    void generateReport(DateRange range,
        int startDay, int startMonth,
        int endDay, int endMonth) {

        System.out.println("From " + range.startDay + "/" + range.startMonth +
            " to " + range.endDay + "/" + range.endMonth);
    }
}
```

```
}  
}
```

JavaScript

Before

```
class ReportService {  
  generateReport(startDay, startMonth, endDay, endMonth) {  
    console.log(  
      `From ${startDay}/${startMonth} to ${endDay}/${endMonth}`  
    );  
  }  
}
```

After

```
class DateRange {  
  constructor(startDay, startMonth, endDay, endMonth) {  
    this.startDay = startDay;  
    this.startMonth = startMonth;  
    this.endDay = endDay;  
    this.endMonth = endMonth;  
  }  
}  
  
class ReportService {  
  generateReport(range, startDay, startMonth, endDay, endMonth) {  
    console.log(  
      `From ${range.startDay}/${range.startMonth} to ${range.endDay}/${range.endMonth}`  
    );  
  }  
}
```

C#

Before

```
class ReportService  
{  
  public void GenerateReport(int startDay, int startMonth, int endDay, int endMonth)  
  {  
    Console.WriteLine(  
      $"From {startDay}/{startMonth} to {endDay}/{endMonth}"  
    );  
  }  
}
```

After


```

class DateRange
{
    public int StartDay;
    public int StartMonth;
    public int EndDay;
    public int EndMonth;

    public DateRange(int startDay, int startMonth, int endDay, int endMonth)
    {
        StartDay = startDay;
        StartMonth = startMonth;
        EndDay = endDay;
        EndMonth = endMonth;
    }
}

class ReportService
{
    public void GenerateReport(DateRange range,
                               int startDay, int startMonth,
                               int endDay, int endMonth)
    {
        Console.WriteLine(
            $"From {range.StartDay}/{range.StartMonth} to {range.EndDay}/{range.EndMonth}"
        );
    }
}

```

Python

Before

```

class ReportService:
    def generate_report(self, start_day, start_month, end_day, end_month):
        print(f"From {start_day}/{start_month} to {end_day}/{end_month}")

```

After

```

class DateRange:
    def __init__(self, start_day, start_month, end_day, end_month):
        self.start_day = start_day
        self.start_month = start_month
        self.end_day = end_day
        self.end_month = end_month

class ReportService:
    def generate_report(self, range_obj,

```

```
        start_day, start_month,
        end_day, end_month):
print(
    f"From {range_obj.start_day}/{range_obj.start_month} "
    f"to {range_obj.end_day}/{range_obj.end_month}"
)
```

Checklist

- Parameter object class introduced first
- Method updated to use parameter object properties
- Call sites updated to construct and pass object
- Parameters removed one at a time (safe delete)
- Tests pass after each parameter removal step
- Final method signature uses only parameter object

7. Extract Interface

Purpose

Extract Interface defines a shared contract that multiple classes can implement. It allows code to depend on **behaviour (what an object does)** rather than **concrete implementation (what it is)**.

Use when:

- Multiple classes offer the same behaviour
 - You want to decouple clients from specific implementations
 - You need polymorphism across different implementations
-

Safe manual mechanics (no automation)

Step-by-step process

1. **Identify the shared behaviour**
 - Look for methods common across multiple classes
 2. **Declare the interface (or abstract base type)**
 - Keep it minimal: only shared method signatures
 3. **Update classes to implement the interface**
 - Ensure each class provides the required methods
 4. **Replace concrete types with the interface where possible**
 - Especially in method parameters and fields
 5. **Run tests after each replacement**
 - Ensure no behavioural change
-

Language considerations

Statically typed languages (Java, C#)

- Interfaces are explicit language constructs
- Classes must declare implementation (implements, :)
- Compiler enforces correctness

Duck-typed languages (JavaScript, Python)

- No enforced interface mechanism by default
- Behaviour is assumed (“if it quacks like a duck...”)

You can still improve design clarity by:

- Documenting expected behaviour
 - Using naming conventions (e.g. *Service, *Provider)
 - Introducing optional structural abstractions:
 - Python: abc.ABC and @abstractmethod
 - TypeScript (if applicable): structural typing with interfaces
-

Python note (ABC for explicit structure)

Python does not require interfaces, but you can introduce one using abc to signal intent.

Example approach:

- Use ABC to define a shared contract
 - Enforce required methods via @abstractmethod
-

Example

Scenario

Multiple notification types exist (EmailNotification, SmsNotification), both support send(message).

We extract a shared interface: Notification.

Java

Before

```
class EmailNotification {  
    void send(String message) {  
        System.out.println("Email: " + message);  
    }  
}
```

```
class SmsNotification {  
    void send(String message) {  
        System.out.println("SMS: " + message);  
    }  
}
```

After

```
interface Notification {  
    void send(String message);  
}
```

```

class EmailNotification implements Notification {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

class SmsNotification implements Notification {
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}

```

JavaScript

Before

```

class EmailNotification {
    send(message) {
        console.log("Email: " + message);
    }
}

class SmsNotification {
    send(message) {
        console.log("SMS: " + message);
    }
}

```

After (duck-typed interface by convention)

```

class EmailNotification {
    send(message) {
        console.log("Email: " + message);
    }
}

class SmsNotification {
    send(message) {
        console.log("SMS: " + message);
    }
}

/*
Interface (informal):
Any object with a send(message) method
*/

```

C#

Before

```
class EmailNotification
{
    public void Send(string message)
    {
        Console.WriteLine("Email: " + message);
    }
}
```

```
class SmsNotification
{
    public void Send(string message)
    {
        Console.WriteLine("SMS: " + message);
    }
}
```

After

```
interface INotification
{
    void Send(string message);
}
```

```
class EmailNotification : INotification
{
    public void Send(string message)
    {
        Console.WriteLine("Email: " + message);
    }
}
```

```
class SmsNotification : INotification
{
    public void Send(string message)
    {
        Console.WriteLine("SMS: " + message);
    }
}
```

Python

Before

```
class EmailNotification:
    def send(self, message):
```

```
print("Email:", message)
```

```
class SmsNotification:  
    def send(self, message):  
        print("SMS:", message)
```

After (using ABC)

```
from abc import ABC, abstractmethod
```

```
class Notification(ABC):  
    @abstractmethod  
    def send(self, message):  
        pass
```

```
class EmailNotification(Notification):  
    def send(self, message):  
        print("Email:", message)
```

```
class SmsNotification(Notification):  
    def send(self, message):  
        print("SMS:", message)
```

Checklist

- Shared behaviour clearly identified
- Interface contains only common operations
- All implementations conform to contract
- Clients depend on interface, not concrete classes
- Python/JS: behaviour is consistently documented or optionally enforced
- Tests pass after abstraction

There Are Many More Refactorings Than These

e.g.

Rename

- Rename parameter
- Rename class
- Rename module/file
- Rename field/property

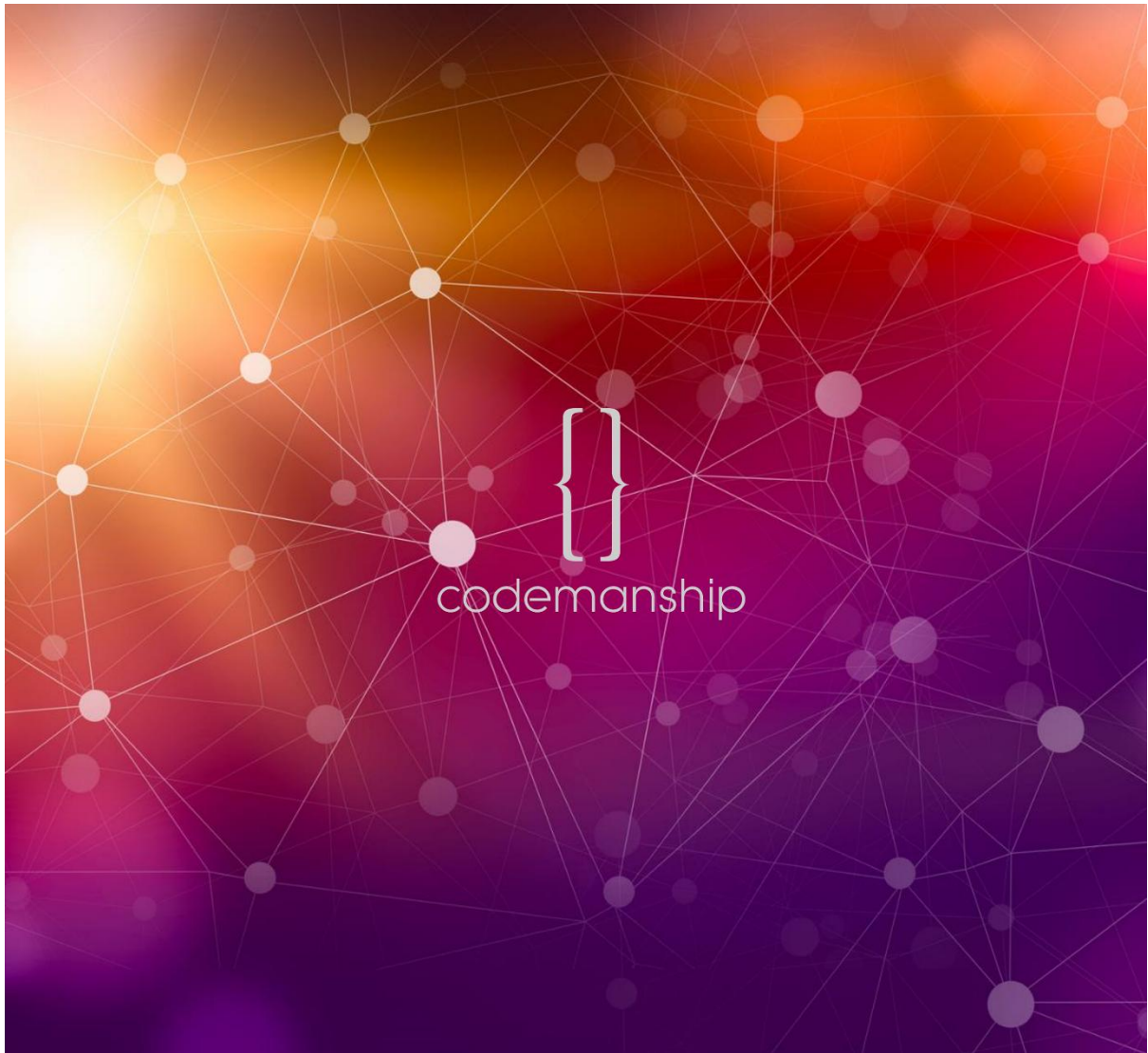
Move

- Move field/property
- Move class
- Move module/package

Introduce

- Introduce variable
- Introduce parameter
- Introduce constant

Visit <https://refactoring.com/catalog/> for a comprehensive list.



www.codemanship.co.uk

jason.gorman@codemanship.com